Python Niveau I — Exercices Jour 2

Module 5 : Fichiers et modules

Exercice 5.1 : Lire un fichier texte [Difficulté : Facile]

Énoncé :

1. Creez un fichier `villes.txt` contenant 5 villes (une par ligne) en utilisant `open()` en mode ecriture
2. Relisez le fichier avec `open()` et `read()` pour obtenir tout le contenu
3. Relisez le fichier avec `readlines()` pour obtenir une liste de lignes
4. Relisez le fichier avec une boucle `for line in f`
5. Utilisez le mot-cle `with` pour une gestion propre du fichier

Indice : `open("fichier", "w")` ouvre en ecriture, `open("fichier", "r")` en lecture. Le bloc `with open(...) as f:` ferme automatiquement le fichier.

▶ Solution

Exercice 5.2 : Écrire dans un fichier [Difficulté : Facile]

Énoncé :

1. Ecrivez une liste de courses dans un fichier `courses.txt` (mode `"w"`)
2. Ajoutez 2 articles supplementaires avec le mode append (`"a"`)
3. Relisez le fichier et affichez le contenu avec les numeros de ligne
4. Ecrivez une liste de nombres (1 a 10) dans un fichier, un par ligne, avec `writelines()`

Indice : Le mode `"w"` ecrase le fichier existant. Le mode `"a"` ajoute a la fin. `writelines()` n'ajoute pas de retour a la ligne automatiquement.

▶ Solution

Exercice 5.3 : Manipuler des CSV [Difficulté : Intermédiaire]

Énoncé :

1. Creez un fichier `employes.csv` avec les colonnes : nom, poste, salaire (5 lignes)
2. Relisez le fichier avec `csv.reader` et affichez chaque ligne
3. Relisez le fichier avec `csv.DictReader` et affichez les noms et salaires
4. Ajoutez un employe avec `csv.writer`
5. Calculez le salaire moyen a partir du fichier

Indice : `import csv`. `csv.reader(f)` retourne un iterateur de listes. `csv.DictReader(f)` retourne un iterateur de dictionnaires. `csv.writer(f).writerow(liste)` ecrit une ligne.

▶ Solution

Exercice 5.4 : Manipuler du JSON [Difficulté : Intermédiaire]

Énoncé :

1. Creez un dictionnaire Python representant un produit (nom, prix, stock, categories)
2. Ecrivez-le dans un fichier `produit.json` avec `json.dump()` (indentation de 2)
3. Relisez le fichier avec `json.load()` et affichez le resultat
4. Convertissez un dictionnaire en chaine JSON avec `json.dumps()`
5. Convertissez une chaine JSON en dictionnaire avec `json.loads()`

Indice : `import json`. `json.dump(obj, f, indent=2)` ecrit dans un fichier. `json.load(f)` lit depuis un fichier. `json.dumps()` et `json.loads()` travaillent avec des chaines.

▶ Solution

Exercice 5.5 : Explorer le système de fichiers [Difficulté : Intermédiaire]

Énoncé :

1. Importez `os` et affichez le repertoire de travail actuel avec `os.getcwd()`
2. Listez les fichiers du repertoire courant avec `os.listdir()`
3. Verifiez si un fichier existe avec `os.path.exists()`
4. Obtenez la taille d'un fichier avec `os.path.getsize()`
5. Separez nom et extension d'un fichier avec `os.path.splitext()`
6. Construisez un chemin avec `os.path.join()`

Indice : `os.path` contient de nombreuses fonctions utiles pour manipuler les chemins. `os.listdir(".")` liste le repertoire courant.

▶ Solution

Exercice 5.6 : Importer et utiliser des modules [Difficulté : Facile]

Énoncé :

1. Importez `math` et utilisez `math.sqrt()`, `math.ceil()`, `math.floor()`, `math.log()`
2. Importez `datetime` et affichez la date du jour, l'heure actuelle, et creez une date specifique
3. Importez `collections` et utilisez `Counter` pour compter les elements d'une liste
4. Montrez les differentes facons d'importer : `import`, `from ... import`, `as`

Indice : `from module import fonction` importe directement. `import module as alias` cree un alias. `collections.Counter(liste)` compte les occurrences.

▶ Solution

Module 6 : NumPy et pandas

Exercice 6.1 : NumPy — Créer des arrays [Difficulté : Facile]

Énoncé :

1. Creez un array NumPy a partir de la liste `[1, 2, 3, 4, 5]`
2. Creez un array de 0 a 19 avec `np.arange()`
3. Creez un array de 10 valeurs uniformement reparties entre 0 et 1 avec `np.linspace()`
4. Creez une matrice 3x4 de zeros, puis une de uns
5. Creez une matrice identite 4x4 avec `np.eye()`
6. Affichez le type, la forme et la dimension de chaque array

Indice : `import numpy as np`. `np.arange(start, stop, step)` est similaire a `range()`. `np.linspace(start, stop, num)` inclut les bornes.

▶ Solution

Exercice 6.2 : NumPy — Opérations vectorisées [Difficulté : Intermédiaire]

Énoncé :

1. Creez un array `a = np.array([10, 20, 30, 40, 50])` et effectuez : `a + 5`, `a * 2`, `a ** 2`
2. Creez un array `b = np.array([1, 2, 3, 4, 5])` et calculez `a + b`, `a * b`, `a / b`
3. Utilisez les comparaisons : `a > 25`, et l'indexation booleenne `a[a > 25]`
4. Calculez les statistiques : `mean()`, `std()`, `min()`, `max()`, `sum()`, `cumsum()`
5. Generez 1000 nombres aleatoires (distribution normale) et calculez moyenne et ecart-type

Indice : NumPy applique les operations element par element (vectorisation). L'indexation booleenne permet de filtrer avec un masque. `np.random.randn(n)` genere une distribution normale.

▶ Solution

Exercice 6.3 : Créer un DataFrame [Difficulté : Facile]

Énoncé :

1. Creez un DataFrame a partir d'un dictionnaire avec les colonnes : `nom`, `age`, `ville`, `salaire`
2. Creez un DataFrame a partir d'une liste de dictionnaires
3. Affichez les 3 premieres lignes avec `head(3)`
4. Affichez les types de colonnes avec `dtypes`
5. Renommez une colonne avec `rename()`

Indice : `import pandas as pd`. `pd.DataFrame(dict)` cree un DataFrame depuis un dictionnaire. `pd.DataFrame(liste_de_dicts)` fonctionne aussi.

▶ Solution

Exercice 6.4 : Charger et explorer [Difficulté : Facile]

Énoncé :

1. Chargez le dataset `tips` de seaborn : `import seaborn as sns; tips = sns.load_dataset("tips")`
2. Affichez les 5 premieres lignes avec `head()`
3. Affichez les informations avec `info()`
4. Affichez les statistiques descriptives avec `describe()`
5. Affichez la forme du DataFrame avec `shape`
6. Affichez les noms de colonnes et les types

Indice : `sns.load_dataset("tips")` charge un DataFrame pret a l'emploi. `info()` affiche les types et les valeurs non-nulles. `describe()` donne les statistiques des colonnes numeriques.

▶ Solution

Exercice 6.5 : Sélection et filtrage [Difficulté : Intermédiaire]

Énoncé : En utilisant le dataset `tips` :

1. Selectionnez la colonne `total_bill` (Series)
2. Selectionnez les colonnes `total_bill` et `tip` (DataFrame)

3. Selectionnez les lignes 10 a 15 avec `iloc`
4. Selectionnez les lignes ou `total_bill > 40`
5. Selectionnez les repas du dimanche (`day == "Sun"`) avec un pourboire > 5
6. Utilisez `query()` pour filtrer et `isin()` pour les jours du week-end

Indice : `df["col"]` retourne une Series. `df[["col1", "col2"]]` retourne un DataFrame. `df.loc[condition]` filtre par condition booleenne. `df.iloc[start:end]` filtre par position.

▶ Solution

Exercice 6.6 : Tri et colonnes calculées [Difficulté : Intermédiaire]

Énoncé : En utilisant le dataset `tips` :

1. Triez par `total_bill` decroissant
2. Triez par `day` puis par `total_bill` croissant
3. Creez une colonne `tip_pct` = tip / total_bill * 100
4. Creez une colonne `categorie` basee sur `total_bill` avec `apply()` (< 15: "Petit", < 30: "Moyen", >= 30: "Gros")
5. Utilisez `map()` pour traduire la colonne `day` en francais

Indice : `sort_values(by, ascending)` trie. `df["col"] = expression` cree une colonne. `apply(fonction)` applique une fonction ligne par ligne. `map(dict)` remplace les valeurs.

▶ Solution

Exercice 6.7 : Merge et concat [Difficulté : Intermédiaire]

Énoncé :

1. Creez deux DataFrames : `employes` (id, nom, dept_id) et `departements` (dept_id, dept_nom)
2. Faites un merge inner sur `dept_id`
3. Faites un merge left pour garder tous les employes (meme sans departement)

4. Faites un merge outer pour tout garder
5. Concatenez deux DataFrames de meme structure avec `pd.concat()`

Indice : `pd.merge(df1, df2, on="cle", how="inner")` fusionne. `how` peut etre `"inner"`, `"left"`, `"right"`, `"outer"`. `pd.concat([df1, df2])` empile verticalement.

▶ Solution

Exercice 6.8 : Groupby et agrégation [Difficulté : Avancé]

Énoncé : En utilisant le dataset `tips` :

1. Calculez le pourboire moyen par jour avec `groupby("day")["tip"].mean()`
2. Calculez plusieurs statistiques par jour avec `agg()`
3. Groupez par `day` et `time`, calculez la moyenne de `total_bill`
4. Utilisez `transform()` pour ajouter la moyenne du groupe a chaque ligne
5. Creez un `pivot_table` : lignes = day, colonnes = time, valeurs = mean(tip)

Indice : `groupby(col)` cree des groupes. `agg({"col": ["mean", "sum"]})` applique plusieurs fonctions. `transform()` retourne un resultat de meme taille. `pd.pivot_table()` cree un tableau croise.

▶ Solution

Module 7 : Visualisation

Exercice 7.1 : Nuage de points [Difficulté : Facile]

Énoncé :

1. Chargez le dataset `tips` avec seaborn
2. Creez un nuage de points : `total_bill` (x) vs `tip` (y)
3. Colorez les points selon le sexe (`sex`)
4. Ajustez la taille des points selon `size`

5. Ajoutez une droite de regression avec `numpy`
6. Ajoutez titre, labels des axes et legende

Indice : `plt.scatter(x, y, c=couleurs, s=tailles)`. Pour la regression : `np.polyfit(x, y, 1)` donne les coefficients, `np.poly1d(coeffs)` cree la fonction.

▶ Solution

Exercice 7.2 : Histogramme [Difficulté : Facile]

Énoncé :

1. Creez un histogramme de la colonne `total_bill` du dataset `tips`
2. Utilisez 20 bins
3. Ajoutez des lignes verticales pour la moyenne (rouge) et la mediane (vert)
4. Ajoutez titre, labels et legende
5. Affichez les valeurs de moyenne et mediane dans la legende

Indice : `plt.hist(data, bins=20)`. `plt.axvline(x, color, linestyle, label)` trace une ligne verticale.

▶ Solution

Exercice 7.3 : Boxplot [Difficulté : Intermédiaire]

Énoncé :

1. Creez un boxplot des pourboires (`tip`) par jour (`day`)
2. Creez un boxplot horizontal des additions par moment (`time`)
3. Personnalisez : couleurs, medianes en rouge, outliers en losange
4. Ajoutez un titre et des labels clairs

Indice : `ax.boxplot(data, labels=labels)`. Pour grouper, preparez les donnees par groupe. `vert=False` pour un boxplot horizontal. `medianprops`, `flierprops` pour personnaliser.

▶ Solution

Exercice 7.4 : Barplot [Difficulté : Intermédiaire]

Énoncé :

1. Creez un barplot simple : pourboire moyen par jour
2. Creez un barplot groupe : pourboire moyen par jour ET par moment (Lunch/Dinner)
3. Ajoutez les valeurs au-dessus de chaque barre
4. Personnalisez les couleurs et le style

Indice : `ax.bar(x, hauteurs)` pour un barplot simple. Pour un barplot groupe, decalez les positions avec `np.arange` et une largeur de barre.

▶ Solution

Exercice 7.5 : Multi-graphiques [Difficulté : Avancé]

Énoncé : Creez un dashboard de 4 graphiques (2x2) a partir du dataset `tips` :

1. (Haut-gauche) Scatter : total_bill vs tip
2. (Haut-droite) Histogramme : distribution de total_bill
3. (Bas-gauche) Boxplot : tip par jour
4. (Bas-droite) Barplot : nombre de repas par jour

Utilisez `fig, axes = plt.subplots(2, 2)` et personnalisez chaque graphique.

Indice : `plt.subplots(2, 2, figsize=(12, 10))` cree une grille 2x2. Accedez aux axes avec `axes[0, 0]`, `axes[0, 1]`, etc. `fig.suptitle()` pour un titre global.

▶ Solution

Exercice 7.6 : Personnalisation avancée [Difficulté : Avancé]

Énoncé : Creez un graphique professionnel montrant l'evolution du pourboire moyen par taille de table :

1. Calculez le pourboire moyen et le nombre d'observations par `size`
2. Creez un graphique en lignes avec marqueurs
3. Ajoutez une zone grisee pour l'ecart-type (fill_between)
4. Annotez le point maximum

5. Ajoutez une grille, une legende soignee et un style professionnel
6. Exportez en PNG haute resolution

Indice : `ax.fill_between(x, y-std, y+std, alpha=0.2)` cree une zone. `ax.annotate("texte", xy, xytext, arrowprops)` ajoute une annotation avec fleche.

▶ Solution